Winter 390 Presentation 4: Team 5

Fauzan Aryaputra, Paris Bozzuti, Ronaldo Tineo, Emma Smith, Aidan Balagtas

March 2026

Project Reflections

All team members

Aidan

- Technical:
  - How CNNs work, how to use Quest/computing cluster, how to optimize a large neural network
- Non-technical:
  - Collaboration with a team on different parts of a broader project, improved ability to present/communicate results, better use of AI to speed up coding workflow and bounce ideas off of
- What do differently:
  - Spend more time understanding math behind model, use GitHub CoPilot (or another IDE-integrated AI) sooner and to a greater extent, establish a clear workflow for group-coding before starting to write/edit code
- Real-world application:
  - Optimizing models and working with AI to do so, working with a team to present results, integrating results and code from other teams into my own work

Emma

- Technical:
  - How to use Quest/computing cluster, skills with GitHub, understanding complex code and integrating new sections
- Non-technical:
  - Standardization of processes and conventions, collaboration across a large group, dividing of tasks, communicating complex results in a clear and concise manner
- What do differently:
  - More coherence with tasks (less segmented work), more collaboration between team members and teams to prevent redundancy
- Real-world application:
  - Integrating analysis within the code of an established process/model, consistency in naming and organization conventions, building off of others and previous work

Fauzan

- Technical:
  - MIL architecture, GitHub-centric workflow, QUEST/cloud computing, turning intuition into a mathematical framework
- Non-technical:
  - Developed collaborative skills, presentation skills
- What do differently:
  - Would work closer with my teammates, was quite segmented since I was only working on the math
- Real-world application:
  - Learned the value of understanding the mathematical foundations of a model - leads to further understanding of how the model can be better, optimized

Ronaldo

- Technical:
  - Analyzing multiple instance learning outputs, including attention and patch-level visualizations, understanding and evaluating how tuning affects model behavior and interpretability, Quest + Github workflow
- Non-technical:
  - Collaboration with team members to translate code/data/visualizations into insights, presenting information clearly, using AI as a tool, task division and skill recognition
- What do differently:
  - More frequent and active communication with team members and other teams, gain a holistic understanding of Github and Quest before doing work to prevent issues
- Real-world application:
  - Working on a long-term project with real life implications
  - Collaboration with teams
  - Importance of organization and streamlining
  - Building off of other’s work and finding ways to solve problems

Paris

- Technical:
  - Working within a shared repository and Quest, basics of multiple instance learning, working with py scripts rather than notebooks, ins and outs of Hyperparameter tuning using Optuna workflow
- Non-technical:
  - Integrating AI into workflow, better practices for presenting model results, task division
- What do differently:
  - More in-person collaboration with team, integrate AI earlier in process, clearer guidelines on using GitHub and Quest, clearer breakdown of what each group member is working on and more coders
- Real-world application:
  - Working with a team on a project of a large scale and long term, jumping into a project that has already been started, learning from other teams and integrating that progress into a bigger solution

Effect of Patch Order on Model Performance

Aidan & Emma

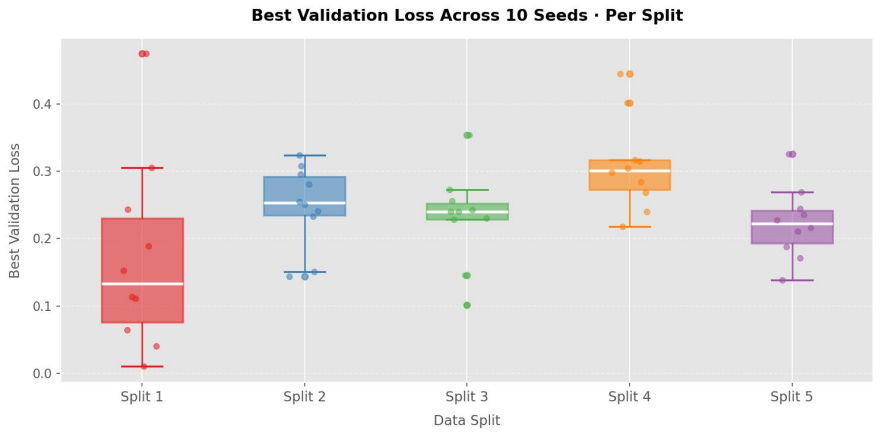
Running model on 10 Seeds

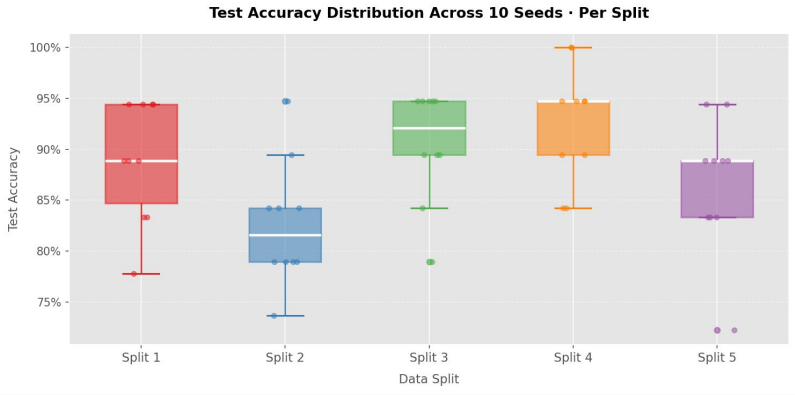
- Random seed affects order of training patches and potentially model performance
- Ran presentation 4 base model on 10 different random seeds
  - 42, 142, 242, 342 … 942
  - Will refer to as seeds 0-9
- Total of 50 models (5 splits x 10 seeds)
  - Overall average test accuracy of **88.3%**, min 72.2% and max 100%
- Relevant results compiled into csv files, on GitHub

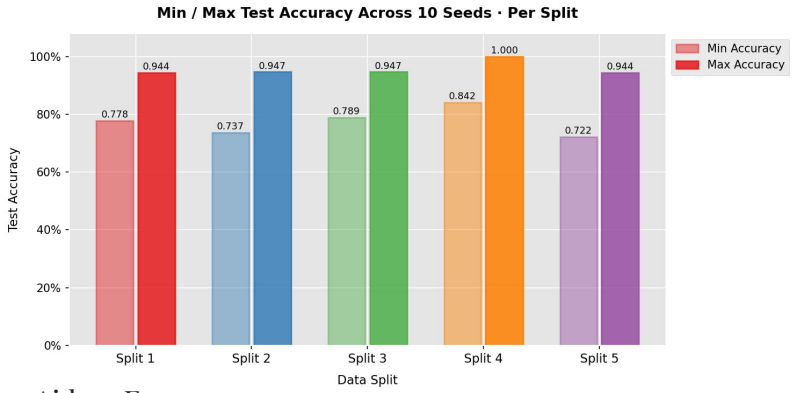
/projects/e32998/STAT390_Winter2026/Team5/Code/Code10_VarSeeds_9Mar_Aidan_Emma

Patch order results in roughly uniform variance of validation loss and test accuracy

- Validation loss of split 1 varies more by seed than other splits
  - Otherwise quite even across splits
- Patch order causes splits to **vary in test accuracy by ~20%**



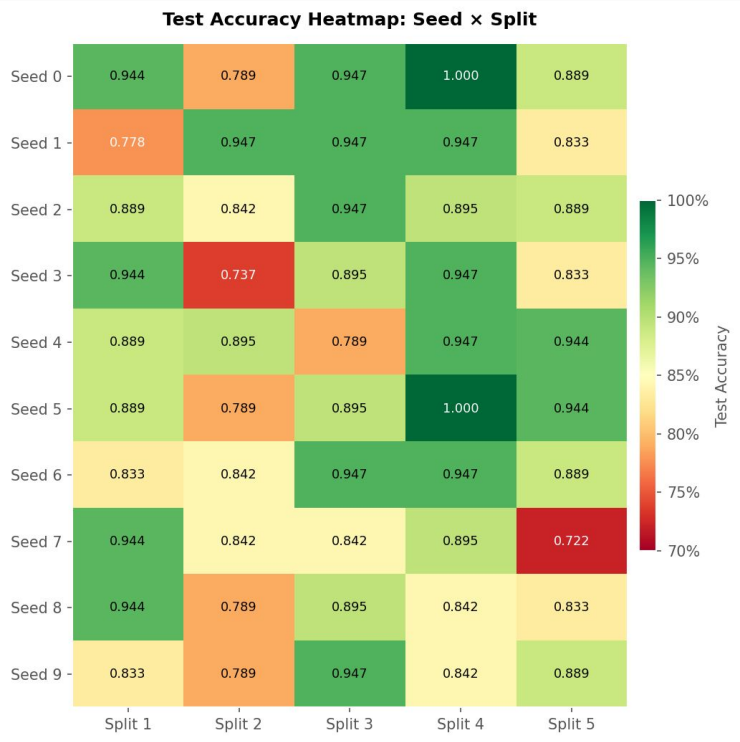




/projects/e32998/STAT390_Winter2026/Team5/Code/Code10_VarSeeds_9Mar_Aidan_Emma

Patch order/seed can systematically affect model test accuracy

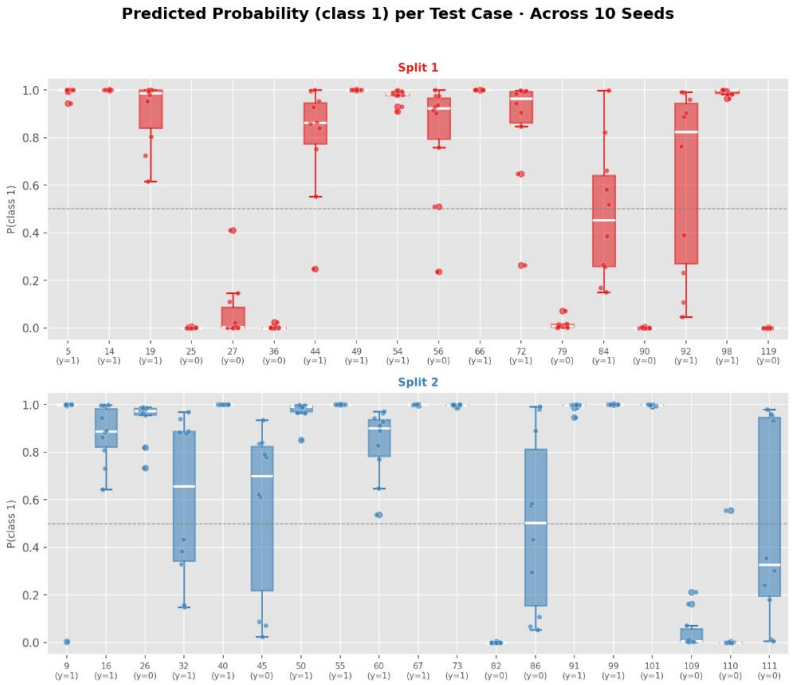
- Some seeds outperform others across all splits
  - Better: seeds 0, 1, 2, 4, 5, 6
  - Worse: seeds 3, 7, 8, 9
- Differences in performance between splits are evident regardless of patch order
  - Better: splits 1, 3, 4
  - Worse: Splits 2, 5

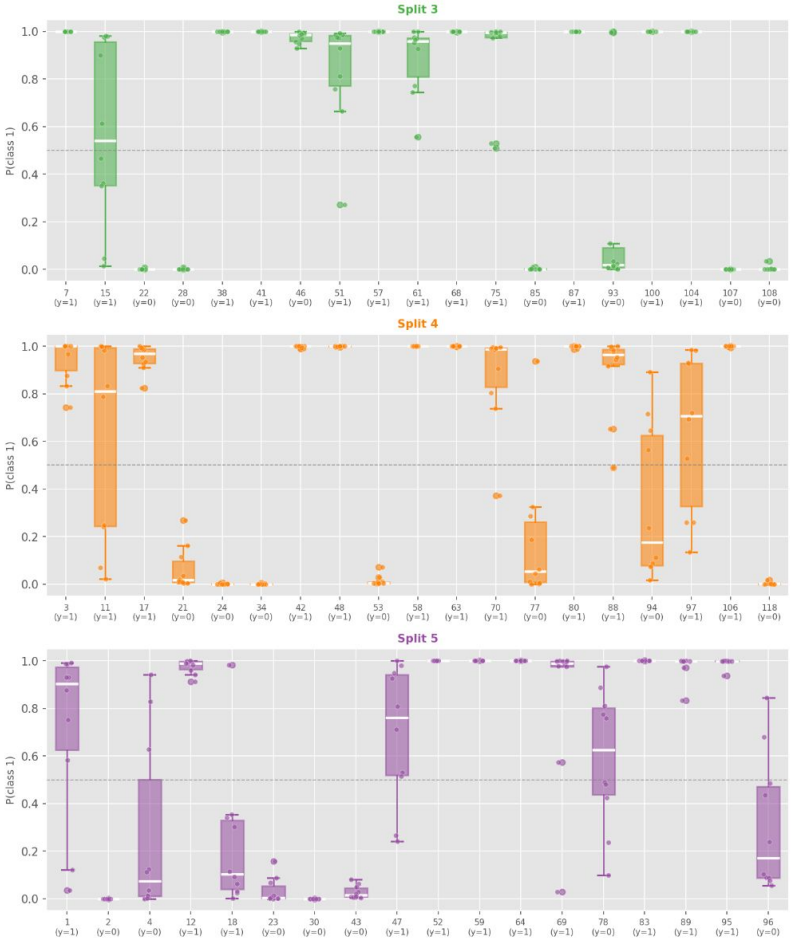


/projects/e32998/STAT390_Winter2026/Team5/Code/Code10_VarSeeds_9Mar_Aidan_Emma

A subset of cases depend on patch order

- Of 93 total cases:
- 15 cases (16.1%) vary in classification by seed
  - 84, 92, 32, 45, 86, 111, 15, 11, 94, 97, 1, 4, 47, 78, 96

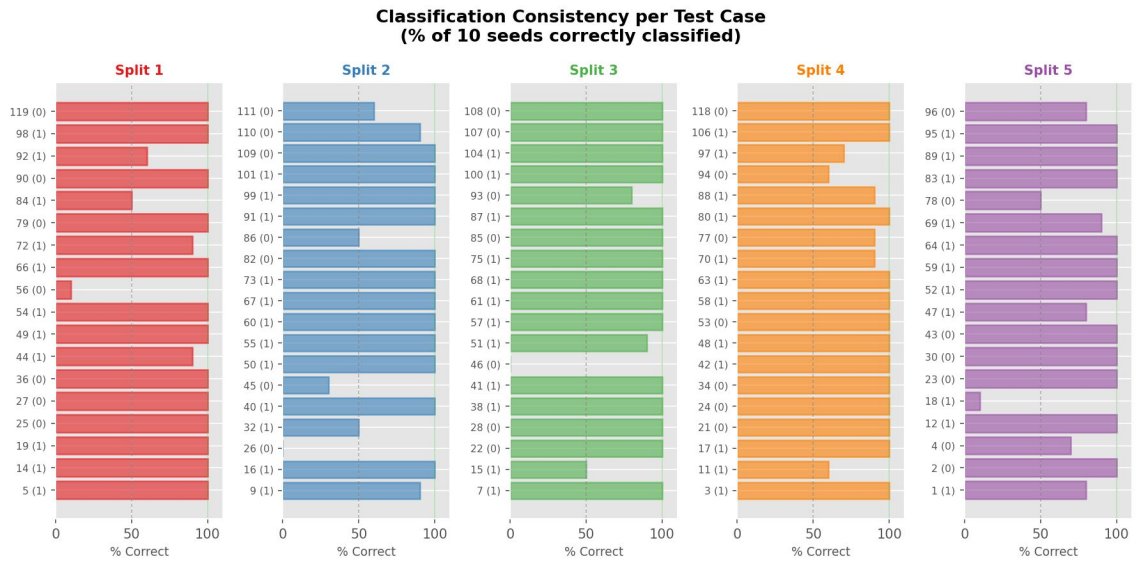




/projects/e32998/STAT390_Winter2026/Team5/Code/Code10_VarSeeds_9Mar_Aidan_Emma

A subset of cases depend on patch order, cont.

- 5 (5.4%) cases almost always misclassified
  - 56, 45, 26, 46, 18
- **10 cases (10.8%) depend on patch order** for correct classification
  - i.e. ~50% correct
  - Benign: 111, 86, 94, 78
  - High-grade: 92, 84, 32, 15, 97, 11
- 78 cases (83.9%) classified correctly independent of patch order



/projects/e32998/STAT390_Winter2026/Team5/Code/Code10_VarSeeds_9Mar_Aidan_Emma

Patch Ordering: Summary/Conclusions

- Patch order results in test accuracy varying by ~20% for all splits
- This is caused by ~16% of cases that change in classification based on patch order, with ⅔ of these cases affected heavily by patch order

/projects/e32998/STAT390_Winter2026/Team5/Code/Code10_VarSeeds_9Mar_Aidan_Emma

Optuna: Hyperparameter Tuning

Paris

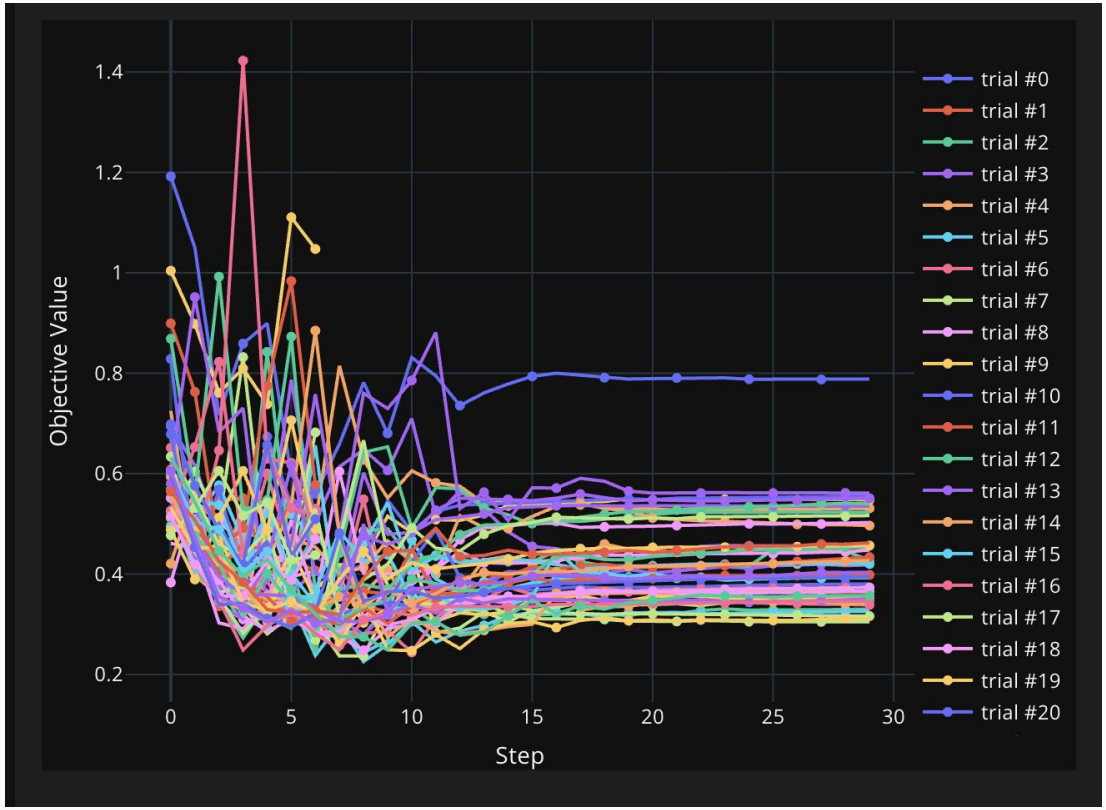
Implementation/Conceptual Issues

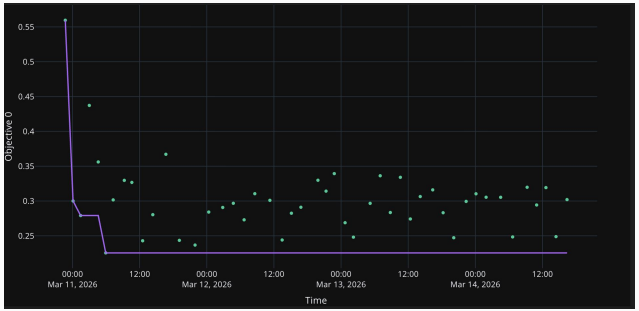
The first issue I ran into was simply a naming convention in the sbatch file, which is that it referred to optuna_tuning.py rather than optuna_training.py.

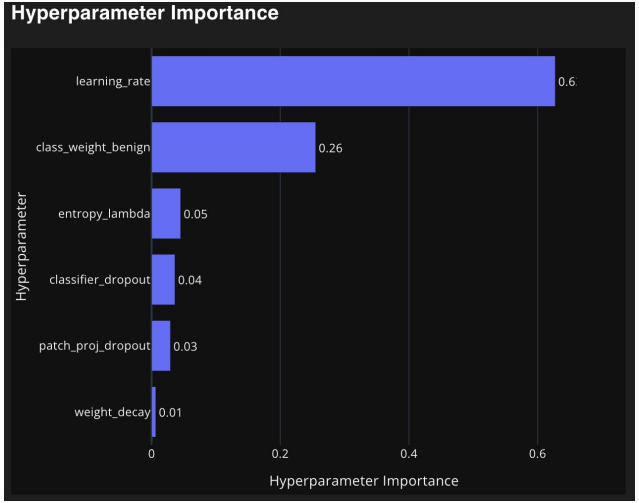
I was unable to get the updated scripts to run without cancelling the job I had already submitted, but I do think I would've liked to change the training as to only run 15 epochs, rather than 30. Most of the training subsided after around there, and the loss was not decreasing.

However, I did not have time to make this adjustment, and the tuning still completed all 64 trials within the timeframe that was set. This did not end up being an issue, because the model was able to train with the optimal hyperparameters very fast since I opted to use GPU. I was still able to test the final model on all 5 data splits.

Dashboard Visualizations







Optimal Values

Parameter	Value
learning_rate	5.225e-05
weight_decay	4.696e-04
patch_proj_dropout	0.511
classifier_dropout	0.198
class_weight_benign	2.137
entropy_lambda	9.976e-04

Best val loss: 0.2255 (mean across 5 folds)

Best Trial (number=5)

0.22549924572712268

Params = [learning_rate: 5.22465277102519e-05, weight_decay: 0.00046962272683261693, patch_proj_dropout: 0.5110095739364624, classifier_dropout: 0.19820380210876465, class_weight_benign: 2.1365861892700195, entropy_lambda: 0.000997635683829176 2]

DETAILS

Code11_OptunaTuning_15Mar_Paris/logs/OPTUNA_5FOLD_MIL_2390999.out

Model Training

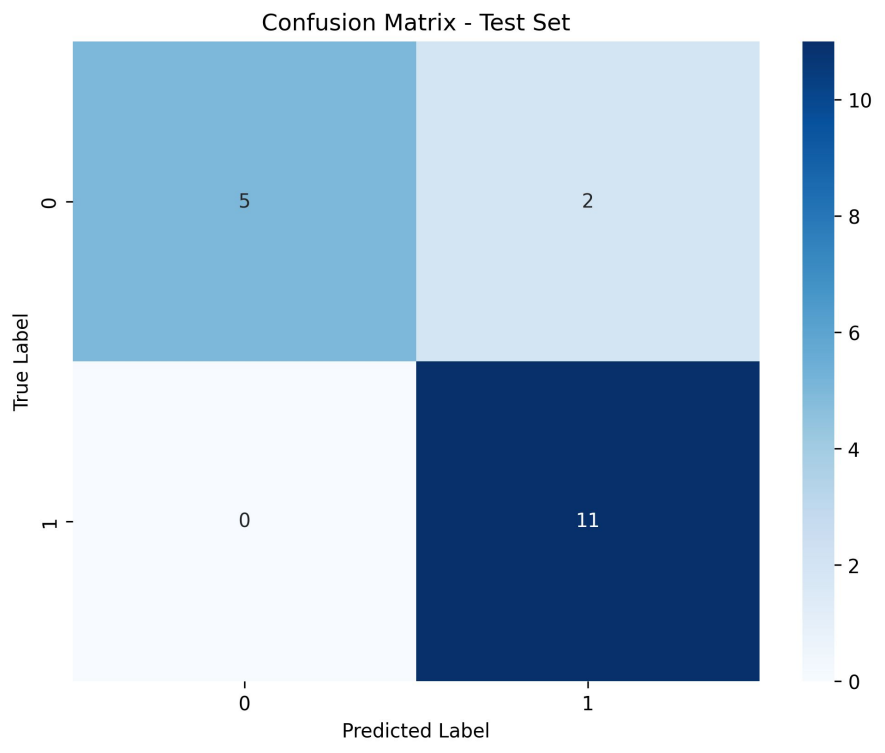
data_splits_new_01.npz

Test Results:
Test Loss: 0.2273
Test Acc: 0.8889
Samples: 18

Early stopping triggered at epoch 19. Best val loss: 0.2277

Best Epoch: 11

Code11_OptunaTuning_15Mar_Paris/logs/MIL_train_2859330.out



projects/e32998/MIL_training/final_runs/run_20260315_171506

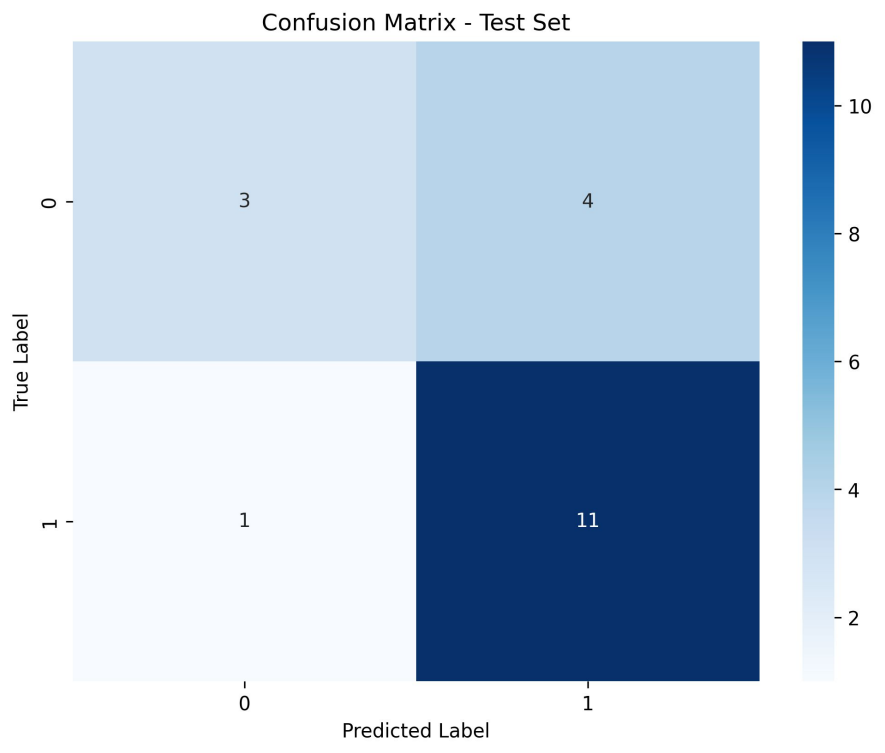
data_splits_new_02.npz

Test Results:
Test Loss: 0.8263
Test Acc: 0.7368
Samples: 19

Early stopping triggered at epoch 16. Best val loss: 0.1674

Best Epoch: 8

Code11_OptunaTuning_15Mar_Paris/logs/MIL_train_2859341.out



projects/e32998/MIL_training/final_runs/run_20260315_171558

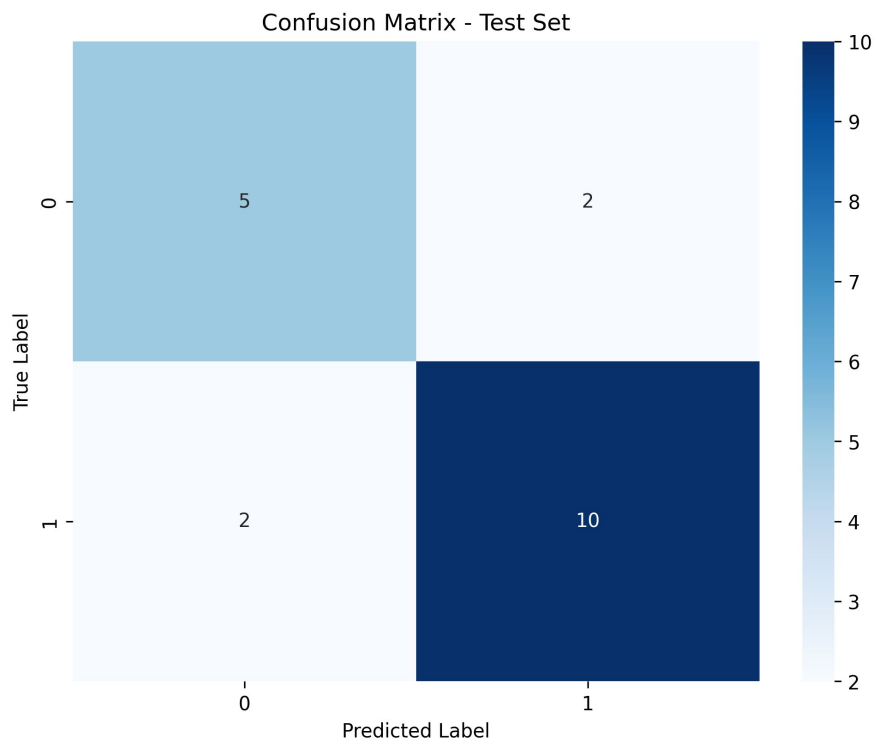
data_splits_new_03.npz

Test Results:
Test Loss: 1.1586
Test Acc: 0.7895
Samples: 19

Early stopping triggered at epoch 16. Best val loss: 0.1936

Best Epoch: 8

Code11_OptunaTuning_15Mar_Paris/logs/MIL_train_2859646.out



projects/e32998/MIL_training/final_runs/run_20260315_175546

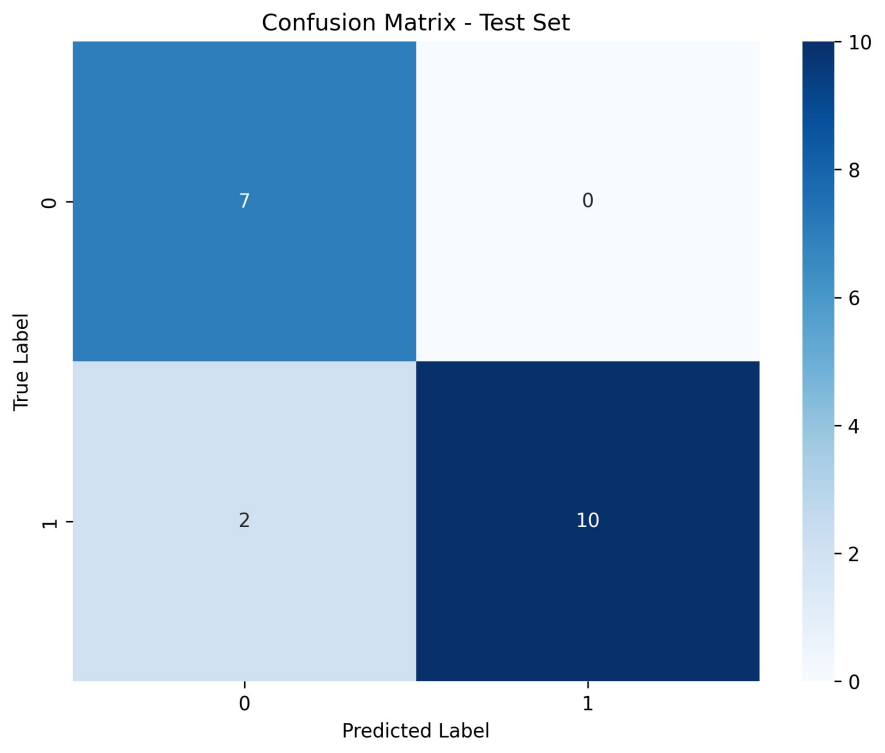
data_splits_new_04.npz

Test Results:
Test Loss: 0.1511
Test Acc: 0.8947
Samples: 19

Early stopping triggered at epoch 13. Best val loss: 0.3823

Best Epoch: 5

Code11_OptunaTuning_15Mar_Paris/logs/MIL_train_2859346.out



projects/e32998/MIL_training/final_runs/run_20260315_171709

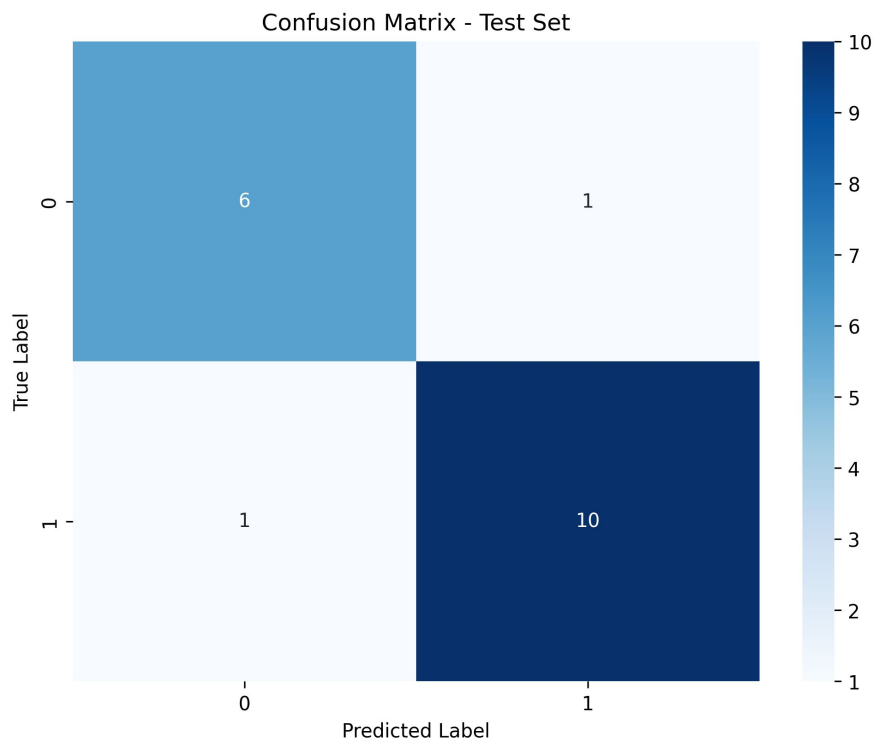
data_splits_new_05.npz

Test Results:
Test Loss: 0.2699
Test Acc: 0.8889
Samples: 18

Early stopping triggered at epoch 16. Best val loss: 0.2033

Best Epoch: 8

Code11_OptunaTuning_15Mar_Paris/logs/MIL_train_2859347.out

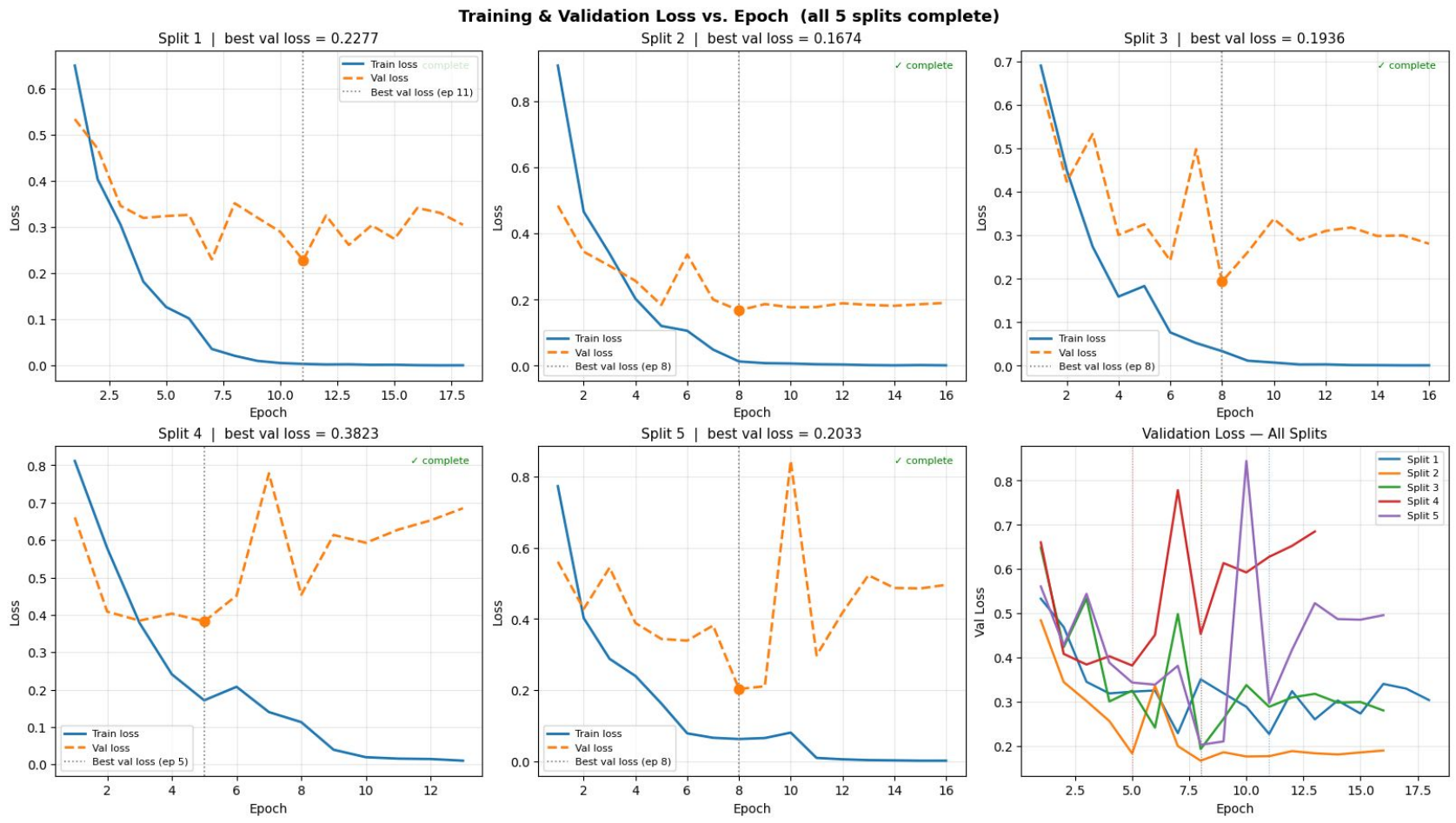


projects/e32998/MIL_training/final_runs/run_20260315_171638

Training

Data Split	Test Loss	Test Accuracy	Misclassified Benign	Misclassified HG	Validation Loss	Early Stopping Epoch
1	0.2273	0.8889	2	0	0.2277	19
2	0.8263	0.7368	4	1	0.1674	16
3	1.1586	0.7895	2	2	0.1936	16
4	0.1511	0.8947	0	2	0.3823	13
5	0.2699	0.8889	1	1	0.2033	16

Loss and Deviation Plots



Code11_OptunaTuning_15Mar_Paris/report.ipynb

Misclassified Cases

Split 1 misclassified (2 cases):

case_id	true_label	predicted_label	prob_class1	deviation	type
56	0	1	0.807626	0.807626	FP
79	0	1	0.779460	0.779460	FP

Split 2 misclassified (5 cases):

case_id	true_label	predicted_label	prob_class1	deviation	type
26	0	1	0.997618	0.997618	FP
32	1	0	0.105643	-0.894357	FN
45	0	1	0.518084	0.518084	FP
109	0	1	0.838805	0.838805	FP
110	0	1	0.981961	0.981961	FP

Split 3 misclassified (4 cases):

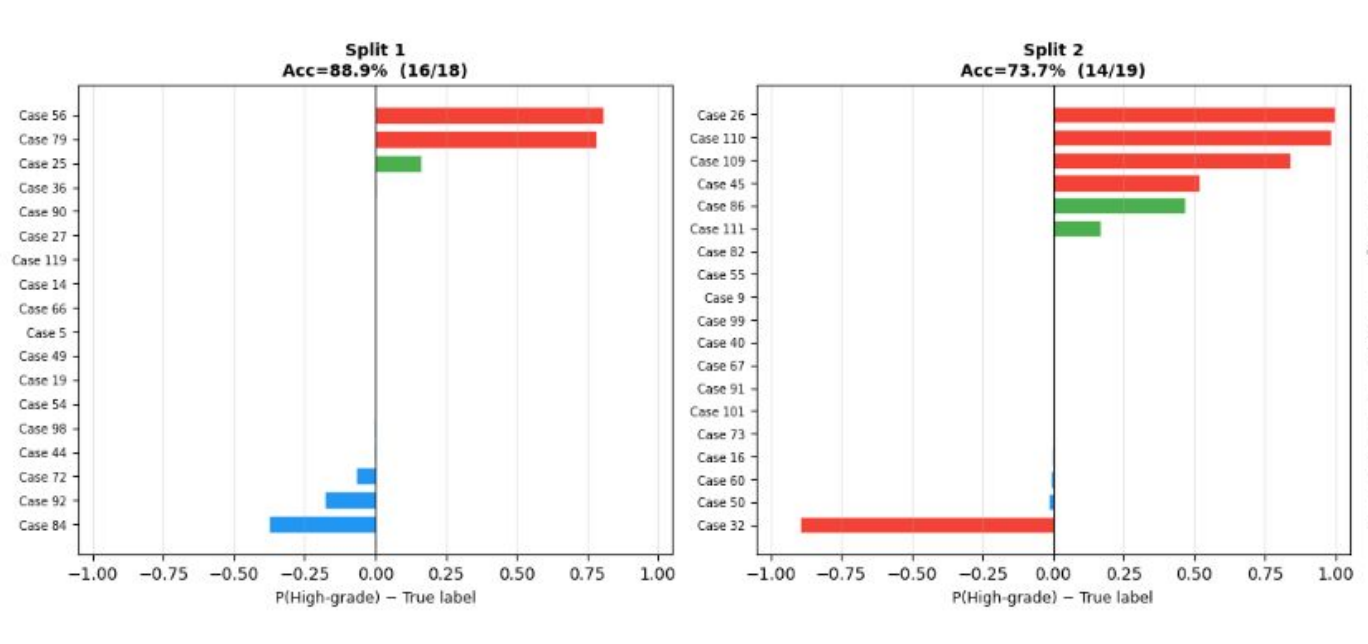
case_id	true_label	predicted_label	prob_class1	deviation	type
15	1	0	0.195183	-0.804817	FN
46	0	1	0.999980	0.999980	FP
51	1	0	0.104491	-0.895509	FN
93	0	1	0.999203	0.999203	FP

Split 4 misclassified (2 cases):

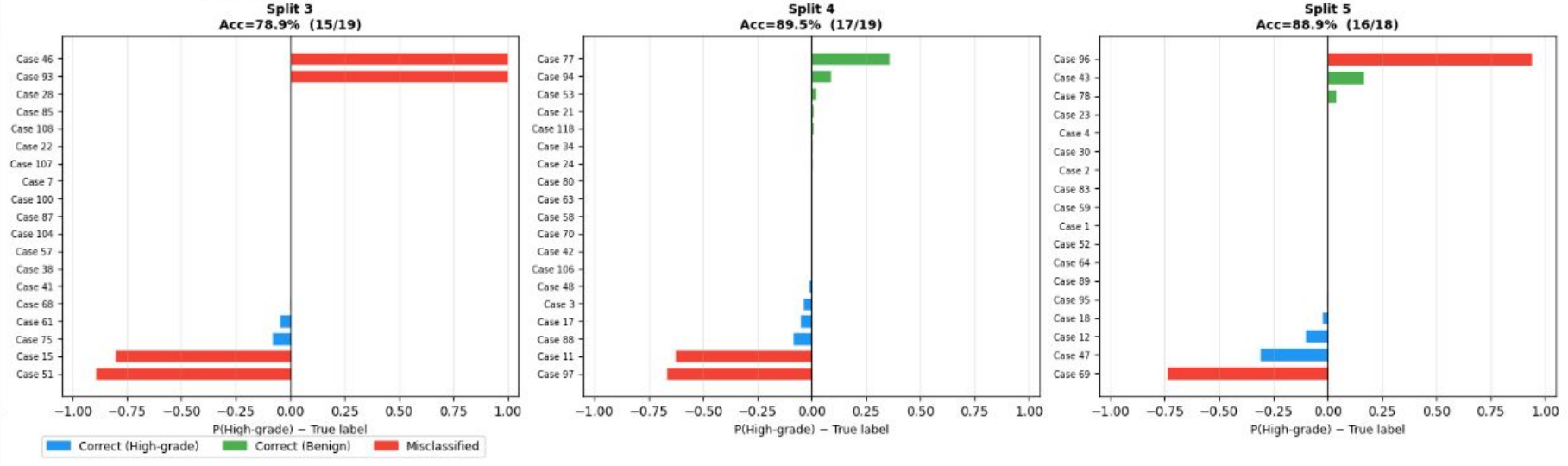
case_id	true_label	predicted_label	prob_class1	deviation	type
11	1	0	0.369320	-0.630680	FN
97	1	0	0.332638	-0.667362	FN

Split 5 misclassified (2 cases):

case_id	true_label	predicted_label	prob_class1	deviation	type
69	1	0	0.261029	-0.738971	FN
96	0	1	0.938150	0.938150	FP



Code11_OptunaTuning_15Mar_Paris/report.ipynb



Code11_OptunaTuning_15Mar_Paris/report.ipynb

Visualizing Effective High Attention Patches for Optuna Tuning (Misclassified Cases)

Ronaldo

Misclassified Cases

Split 1 misclassified (2 cases):

case_id	true_label	predicted_label	prob_class1	deviation	type
56	0	1	0.807626	0.807626	FP
79	0	1	0.779460	0.779460	FP

Split 2 misclassified (5 cases):

case_id	true_label	predicted_label	prob_class1	deviation	type
26	0	1	0.997618	0.997618	FP
32	1	0	0.105643	-0.894357	FN
45	0	1	0.518084	0.518084	FP
109	0	1	0.838805	0.838805	FP
110	0	1	0.981961	0.981961	FP

Split 3 misclassified (4 cases):

case_id	true_label	predicted_label	prob_class1	deviation	type
15	1	0	0.195183	-0.804817	FN
46	0	1	0.999980	0.999980	FP
51	1	0	0.104491	-0.895509	FN
93	0	1	0.999203	0.999203	FP

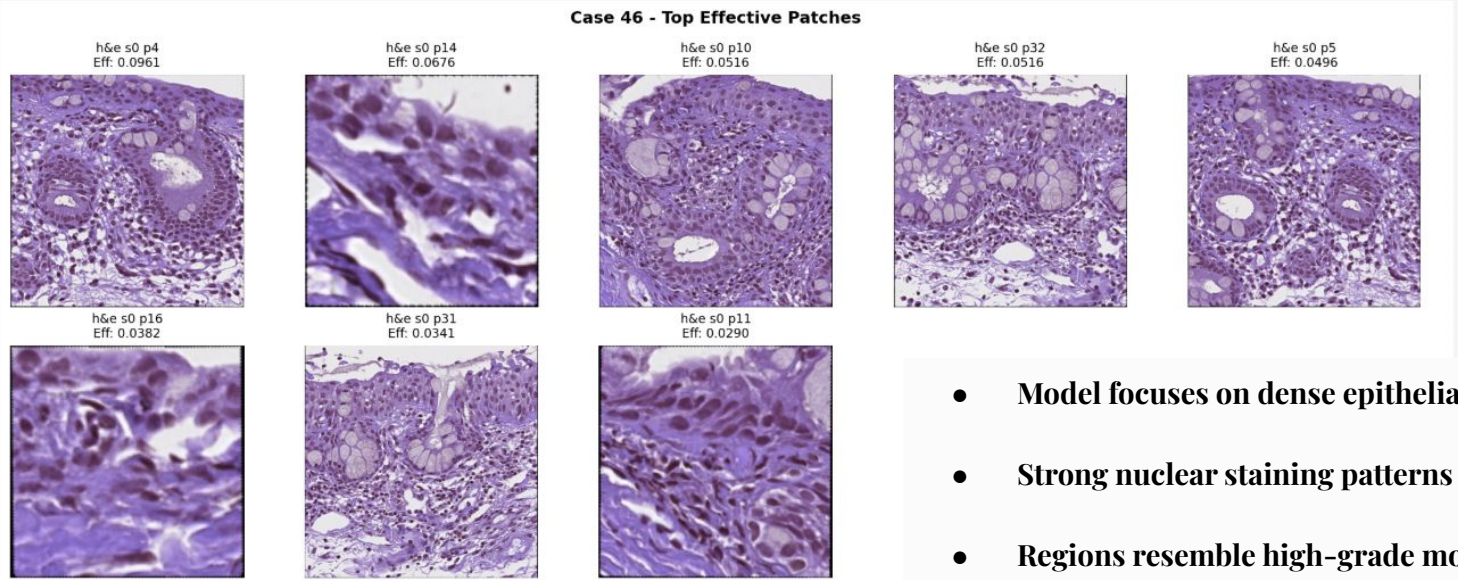
Split 4 misclassified (2 cases):

case_id	true_label	predicted_label	prob_class1	deviation	type
11	1	0	0.369320	-0.630680	FN
97	1	0	0.332638	-0.667362	FN

Split 5 misclassified (2 cases):

case_id	true_label	predicted_label	prob_class1	deviation	type
69	1	0	0.261029	-0.738971	FN
96	0	1	0.938150	0.938150	FP

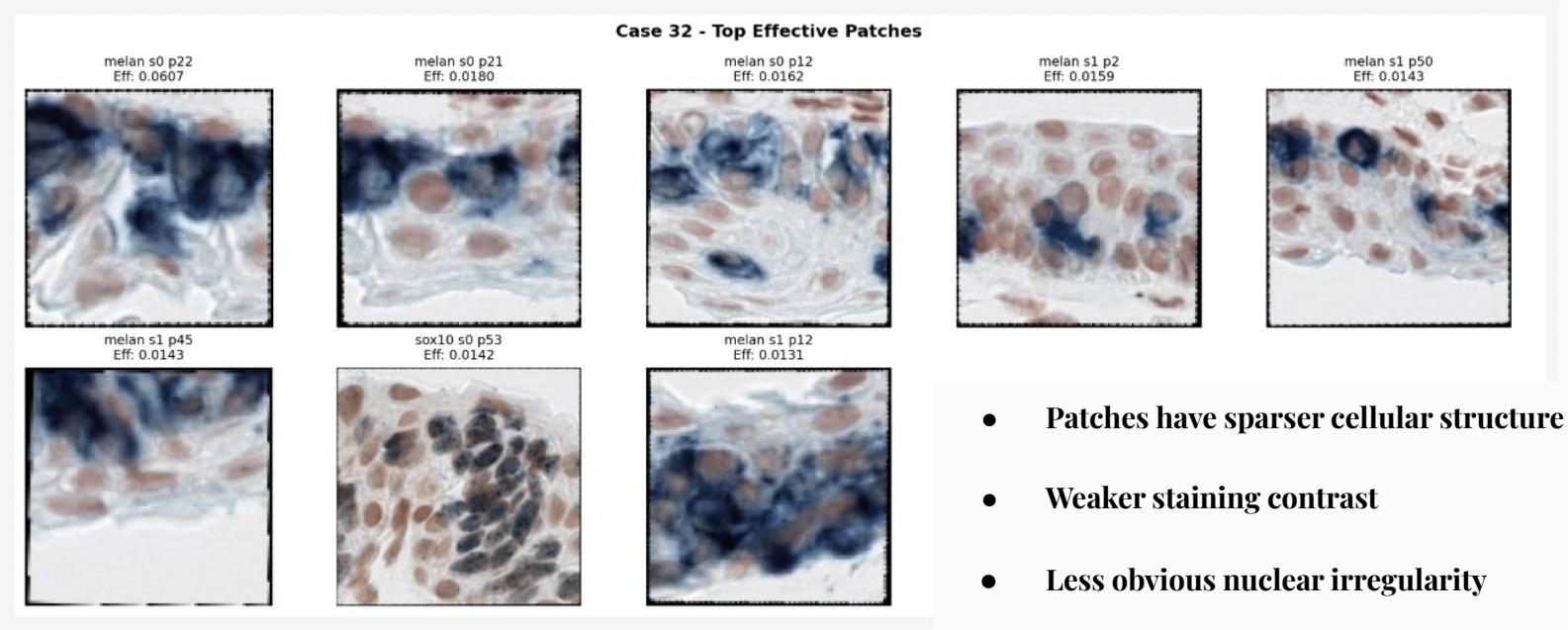
Case 46 (False Positive)



- **Model focuses on dense epithelial regions**
- **Strong nuclear staining patterns**
- **Regions resemble high-grade morphology**

projects/e32998/MIL_training/final_runs/run_20260315_175546

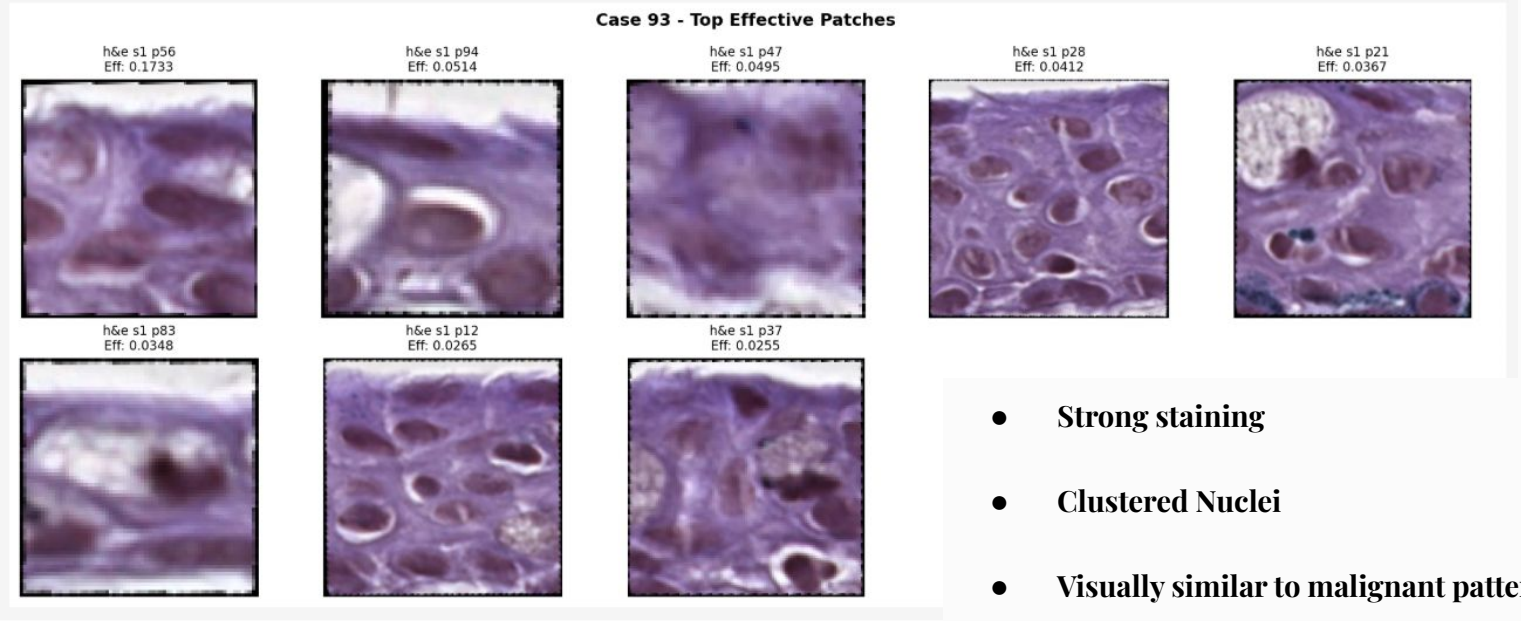
Case 32 (False Negative)



- **Patches have sparser cellular structure**
- **Weaker staining contrast**
- **Less obvious nuclear irregularity**

projects/e32998/MIL_training/final_runs/run_20260315_171558

Case 93 (False Positive)



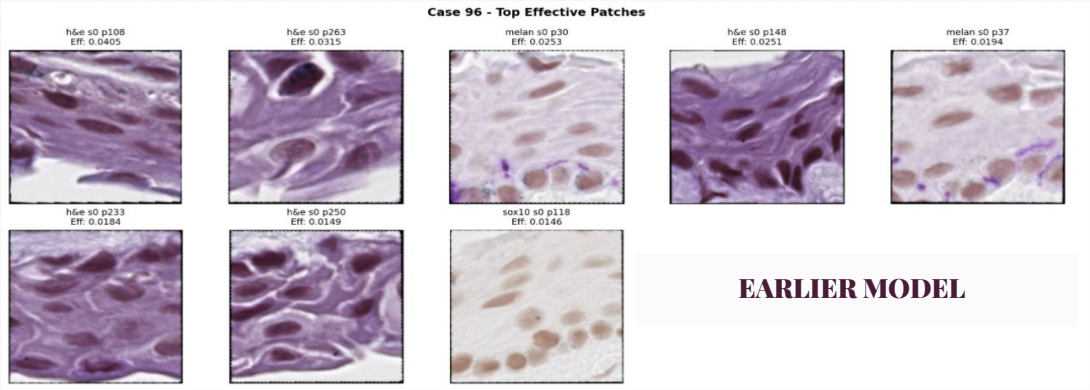
- **Strong staining**
- **Clustered Nuclei**
- **Visually similar to malignant patterns**

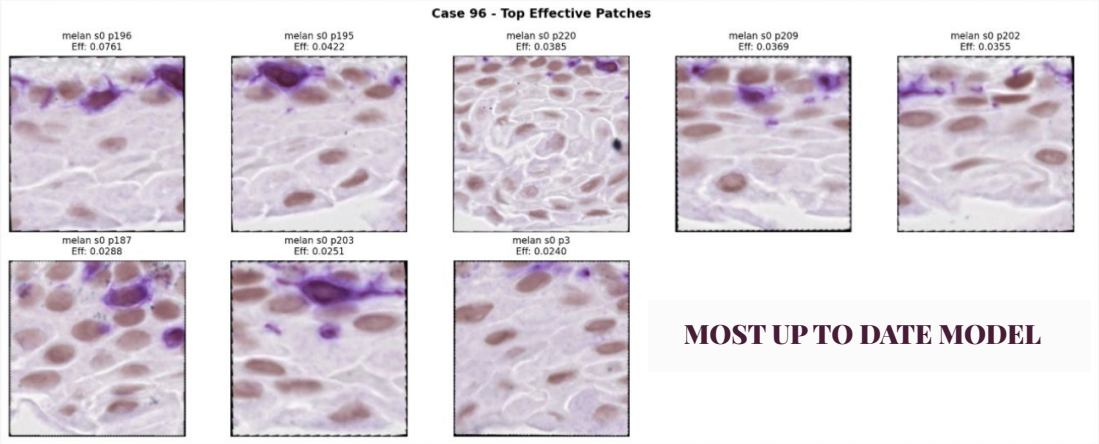
projects/e32998/MIL_training/final_runs/run_20260315_175546

Insights from Misclassified Cases

- Model focuses heavily on nuclear density and staining intensity
  - Dense benign regions can resemble malignant patterns
- Subtle malignant regions may be overlooked if attention focuses elsewhere

Effect of Optuna Hyperparameter Tuning





Earlier model attends to broader stromal regions

New model focuses on localized cellular structures

New model shows stronger peak attention weights

Theory: Small Data Memorization in MIL

Fauzan

Problem Statement

- **Goal**: Walk through why an attention-based MIL model can exhibit overfitting in a small-data paradigm, even when each case contains many patches

- **Cases**:
  1) Overparameterized supervised learning
     a) How does an overly expressive model memorize small training sets and drive training loss to zero, while still having weak generalizability?
  2) Increasing the number of patches in MIL
     a) Why does there remain a small-data problem even as the number of patches tends toward infinity? What are the implications of this approach?
  3) MIL memorization
     a) How does the MIL structure still have the ability to memorize the training data? How does this interact with increasing the number of patches?

Case 1: Supervised Overparameterized Model

Setup

- Let the training set of size n be:

$$S = \{(x_i, y_i)\} \qquad y_i \in \{0, 1\}$$

- With a model denoted:

$$f_\theta(x) \qquad \theta \in R^p$$

  - Where:

$$p \gg n$$

- We train by empirical risk minimization:

$$\hat{\theta} \in argmin_\theta \hat{R}(\theta) \qquad \hat{R}(\theta) = \frac{1}{n}\sum^{n} loss(f_\theta(x_i), y_i)$$

Case 1: Supervised Overparameterized Model (cont.)

Why training loss approaches zero when the model is overparameterized

Memorization

- If the model is overparameterized (i.e., has too many parameters with respect to training data), there often exists a set of parameters θ such that, for all i:

$$f_\theta(x_i) = y_i$$

  - Or for cross-entropy loss, the correct class logit can be made much larger than the other logit on each training point

- Hence, this drives train error down to zero
  - This is because the model has too many degrees over which it can fit over a relatively smaller training dataset
  - The model finds an interpolating solution on the training cases

Case 1: Supervised Overparameterized Model (cont.)

Why overparameterization does not imply good test performance

- The population risk (i.e., true out-of-sample average error) is denoted by:

$$R(\theta) = E_{(X,Y)\sim P}[loss(f_\theta(X), Y)]$$

  - With the generalization gap represented by:

$$R(\theta) - \widehat{R}(\theta)$$

- A small training loss due to overparameterization only tells us the empirical risk is small; it says nothing about population risk

Case 1: Supervised Overparameterized Model (cont.)

A standard example that shows memorization

- Say we have a model that is overparameterized to the point where it can isolate individual training points, so we can define:

$$f(x) = \sum_{i=1}^{n} y_i \phi_i(x) = y_1 \phi_1(x) + y_2 \phi_2(x) + \ldots$$

  - Where phi is an indicator function:

$$\phi_i(x_j) = \{1,\ j = i\ ;\ 0,\ j \neq i\}$$

- This means that:

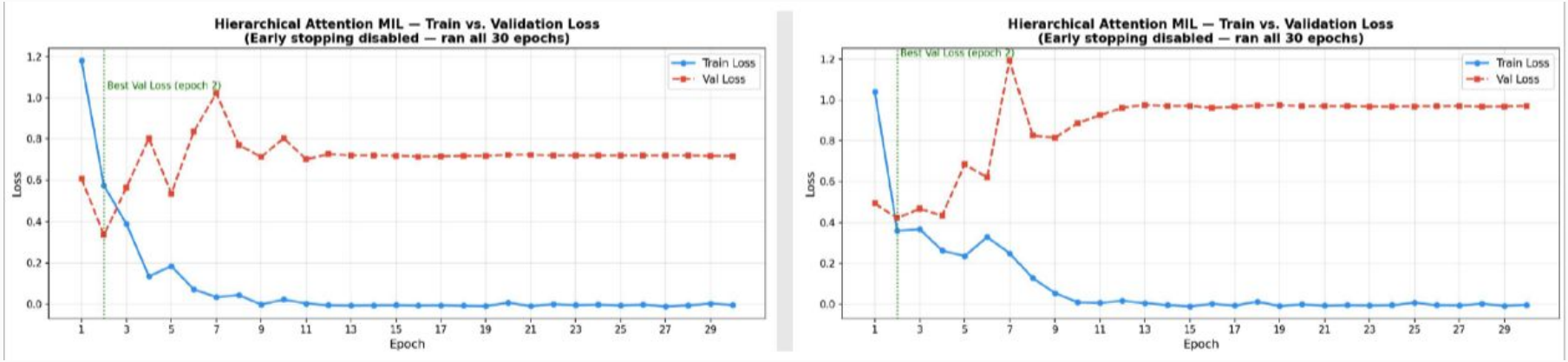
$$f(x_i) = y_i$$

  - Or in other words, the training set is fit perfectly

- But away from the training points (e.g., on out-of-distribution points), the model is unconstrained
  - Therefore, exact interpolation does not imply low test error

Case 1: Supervised Overparameterized Model (cont.)

How this relates to the loss curves



- Training loss drops rapidly toward zero, while validation loss stays high, noisy, or even worsens
  - Within our mathematical framework, this is exactly our definition of training data memorization:

$$\widehat{R}(\theta) \to 0, \; while \; R_{val}(\theta) \nrightarrow 0$$

Case 2: MIL Small-Data Argument

Setup

- Each supervised sample is a case B_i, where B_i is a bag of patches:

$$(B_i, y_i),\ i = 1, \ldots, n \qquad B_i = \{x_{i1}, x_{i2}, \ldots, x_{im_i})$$

- The model predicts at the case level:

$$\hat{y}_i = F_\theta(B_i)$$

- Thus, training loss remains:

$$\hat{R}(\theta) = \frac{1}{n}\sum^{n} loss(f_\theta(B_i), y_i)$$

Case 2: MIL Small-Data Argument (cont.)

Supervision is only at the bag level, so the problem remains small-data

Small-Data

- There are no patch labels, only one label per case
- The number of supervised constraints is only n, not the sum of all patches across all cases
- Thus, even if:

$$m_i \to \infty$$

  - The objective remains:

$$\widehat{R}(\theta) = \frac{1}{n}\sum^{n} loss(f_\theta(B_i), y_i)$$

  - Which contains n supervised terms

Case 2: MIL Small-Data Argument (cont.)

More patches do not create more independent supervised samples

- "If a case has a very large number of patches, is that like having a very large number of training examples?"
  - No, because all *m* patches share the same case label
  - Patches are jointly aggregated into one prediction

- Mathematically, the patches within a case are conditionally dependent components of a single supervised observation:

$$(B_i, y_i)$$

- Thus, the independent units for statistical learning are the labeled cases:

$$(B_1, y_1), \ldots, (B_n, y_n)$$

  - And not the individual patches inside each case

- So **effective sample size** for supervision is n, not $\sum_i m_i$

Case 2: MIL Small-Data Argument (cont.)

Statistical framework for the small-data argument

- Suppose cases are i.i.d.:

$$(B_i, y_i) \sim i.i.d.\ P_{case}$$

  - Then generalization bounds scale with the number of independent draws from the population of all cases (i.e., *n*)
  - They do not scale with the number of patches per case
    - Unless patch labels are observed independently and used as supervision, which they are not

- Thus, asymptotically generalization error shrinks as a function of *n*:

$$O\left(\frac{1}{\sqrt{n}}\right)$$

  - And not *m*:

$$O\left(\frac{1}{\sqrt{\sum_i m_i}}\right)$$

Case 2: MIL Small-Data Argument (cont.)

Big O Derivation

- Define the loss on a single case:

$$Z_i = loss(f_\theta(B_i), y_i)$$

- Recall the generalization gap:

$$R(\theta) - \widehat{R}(\theta) \qquad R(\theta) = E[Z_i] \qquad \widehat{R}(\theta) = \frac{1}{n}\sum^{n} Z_i$$

- Continue the assumption that cases are i.i.d. and loss has finite variance:

$$Var(Z_i) = \sigma^2 < \infty$$

- The variance of the sample mean is:

$$Var\left(\frac{1}{n}\sum_{i}^{n} Z_i\right) = \frac{\sigma^2}{n}$$

Case 2: MIL Small-Data Argument (cont.)

Big O Derivation (cont.)

- So the standard deviation is:

$$\frac{\sigma}{\sqrt{n}}$$

- Thus, the absolute difference between training error and expected error shrinks like:

$$O\left(\frac{1}{\sqrt{n}}\right)$$

  - Because the sample average fluctuates around the expected value by the standard deviation

Case 2: MIL Small-Data Argument (cont.)

Intuition behind why adding more patches does not increase generalizability

- If we increase the number of patches substantially, it is tempting to think that we are greatly increasing the amount of training data we have
  - Instead, we should frame it as seeing each case in greater detail
  - It helps the model learn what **that specific case** looks like with regards to its label, not what the population of cases with that label looks like

- Suppose each case *i* has an underlying patch distribution P_i, and patches are samples from that case-specific distribution:

$$x_{ij} \sim P_i$$

  - Then as m_i approaches infinity, the model can learn P_i very well
  - But it only learns about case *i*

- To generalize better, what matters is learning the relationship between:

$$P_i \rightarrow y_i$$

  - Across many different cases *i*

Case 3: Overparameterized MIL Memorization

Memorization is still possible in the MIL framework

- If the model is sufficiently expressive, it can learn a bag-level function that identifies the finite training bags:

$$f_\theta(B_i) = y_i$$

- This can happen in several ways:
  1. Case-specific fingerprint memorization
     - The model may learn embedding and/or attention patterns that are unique to each training case
  2. Misleading patch subsets
     - For each high-grade training case, it may attend to a small set of common but non-generalizable patches specific to that case
  3. Attention + Classifier memorization
     a. The attention + classifier can map each training bag representation to the correct label, even if what it is learning internally is not biologically relevant

Case 3: Overparameterized MIL Memorization (cont.)

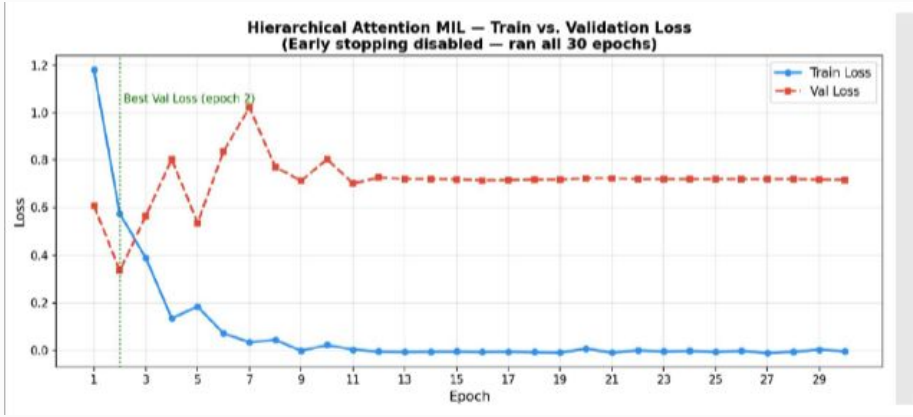
Why adding more patches may make memorization easier

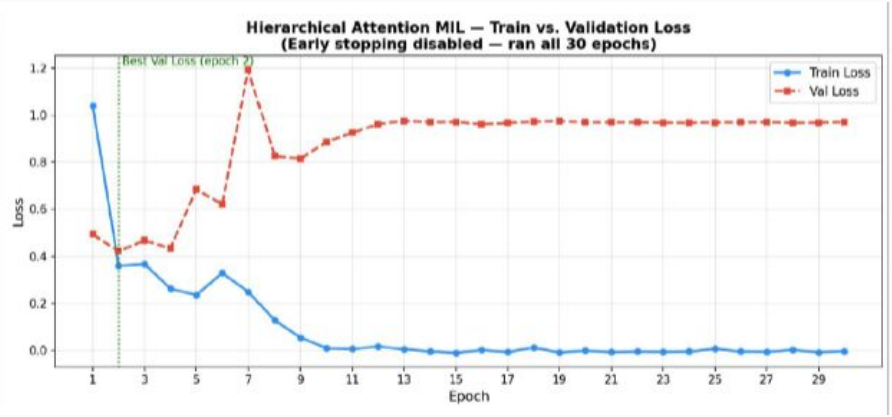
- Combinatorial flexibility argument
  - More patches per case can increase the flexibility of the bag representation combinatorially
  - The model searches over many candidate subsets through attention
  - As the number of patches grows, there are more possible attention patterns, and thus, more chances to find case-specific misleading features
  - Increasing patch count can increase model flexibility while effective supervised sample size remains *n*, worsening overfitting risk
- Extreme value argument
  - If we increase patches substantially, then the maximum attention score for a bag of patches tends to increase with m, even when scores are not meaningful
    - The more patches we inspect, the higher the probability that one patch looks “important” when it is not
  - This increases the chance of memorizing noise in training cases without learning a generalizable disease signal

Case 3: Overparameterized MIL Memorization (cont.)

Why adding more patches may make memorization easier (cont.)

- Within-case vs between-case distinction
  - Increasing the number of patches helps with giving a more accurate estimate of the internal distribution of patches within case *i*
  - But this may make memorization easier, because the model can form a more precise understanding of each training case
  - Instead of learning general disease signal, it may learn the case-specific bag signature's relationship to the label





Assumptions

1) Cases are the independent supervised units
2) Patch labels are not observed
3) MIL architecture has enough capacity to fit the training set nearly perfectly
4) Patches within a case are not counted as independent supervised samples
5) Validation cases come from the same population

Core Idea

- The unit of supervision is the case, not the patch
  - So even if we increase the patch count substantially, the number of independent supervised observations is still the number of labeled cases (<100)
- Overfitting
  - Increasing number of patches in the MIL framework increases the risk of overfitting
- Effective sample size
  - Driven primarily by number of labeled cases, not patches
- Generalization
  - Adding more patches per case may reduce within-case error, but does not solve the fundamental small-data generalization problem